



PLANTS VS ZOMBIE: GOLDEN SPATULA

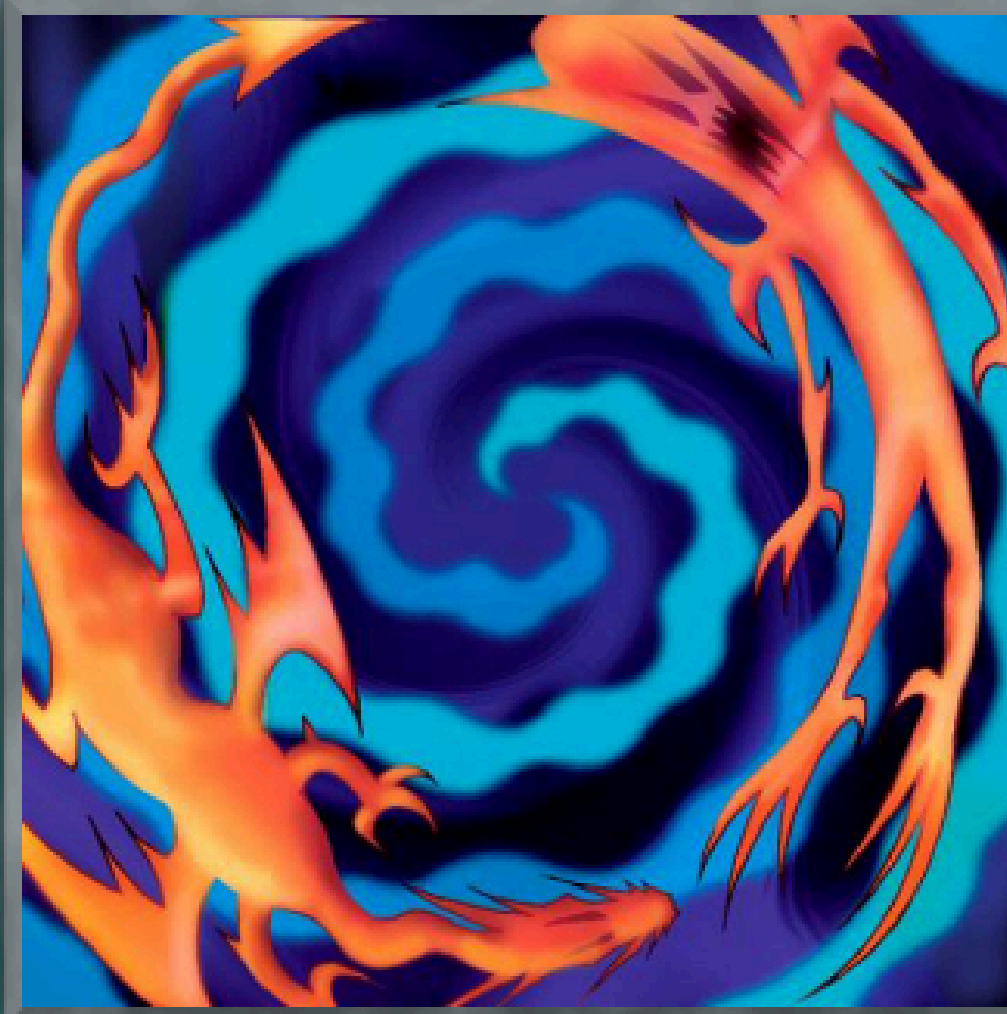




POLYMERIZATION



[SPELL CARD]



Fusion Summon 1 Fusion Monster from your Extra Deck, using monsters from your hand or your side of the field as Fusion Materials.

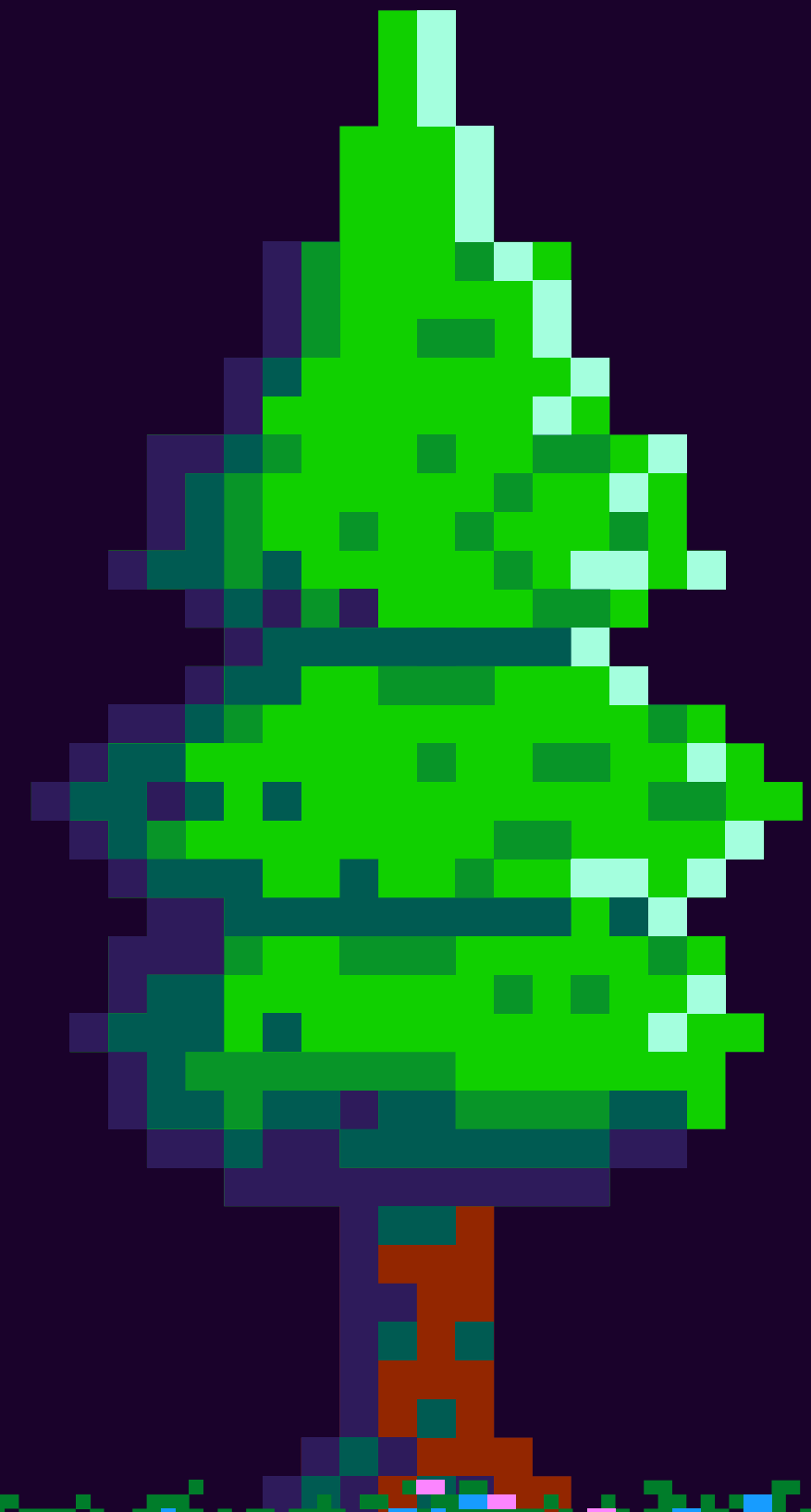
©1996 KAZUKI TAKAHASHI



HOW TO PLAY OUR GAME?



WHAT
CHARACTERS
WE USE?



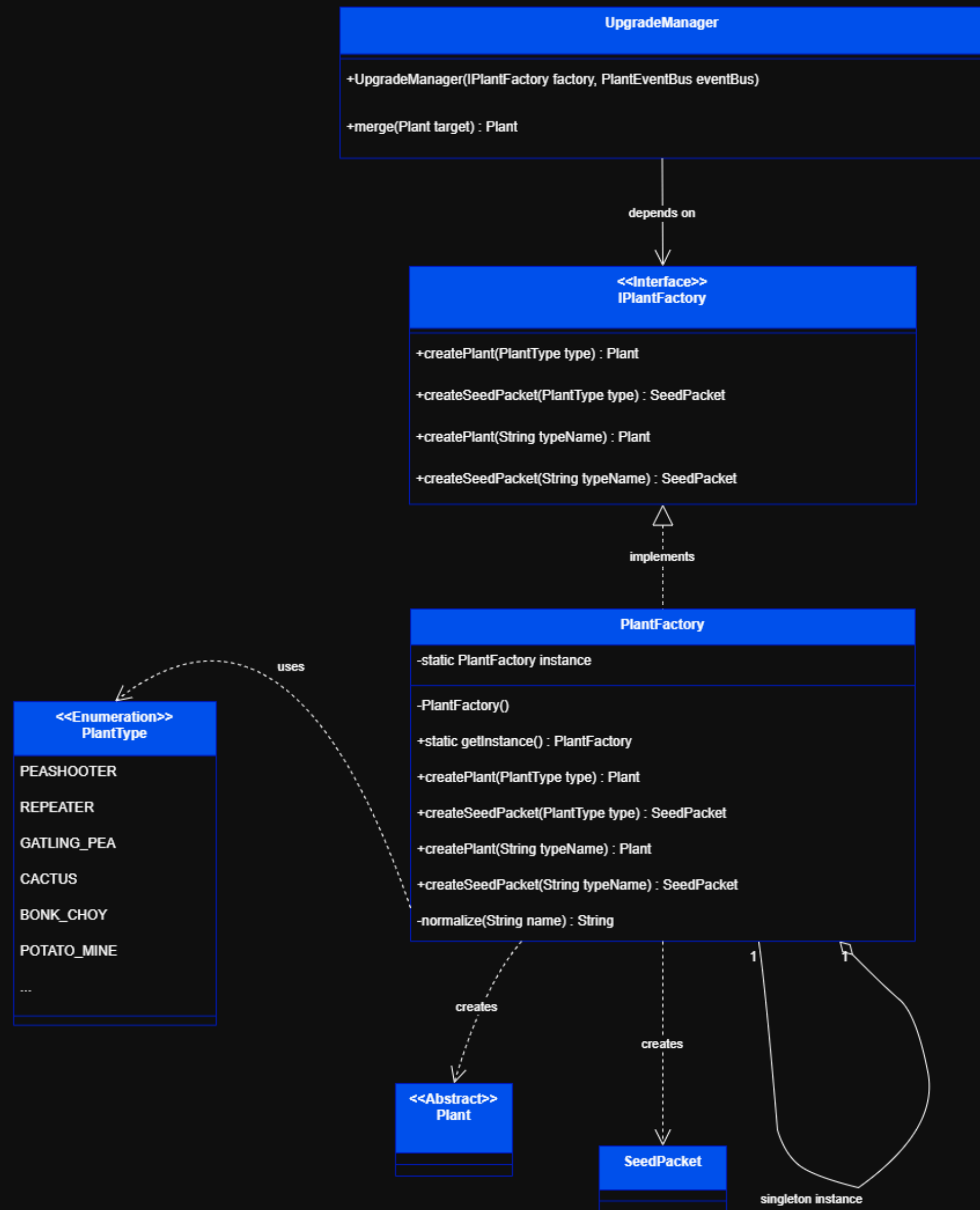




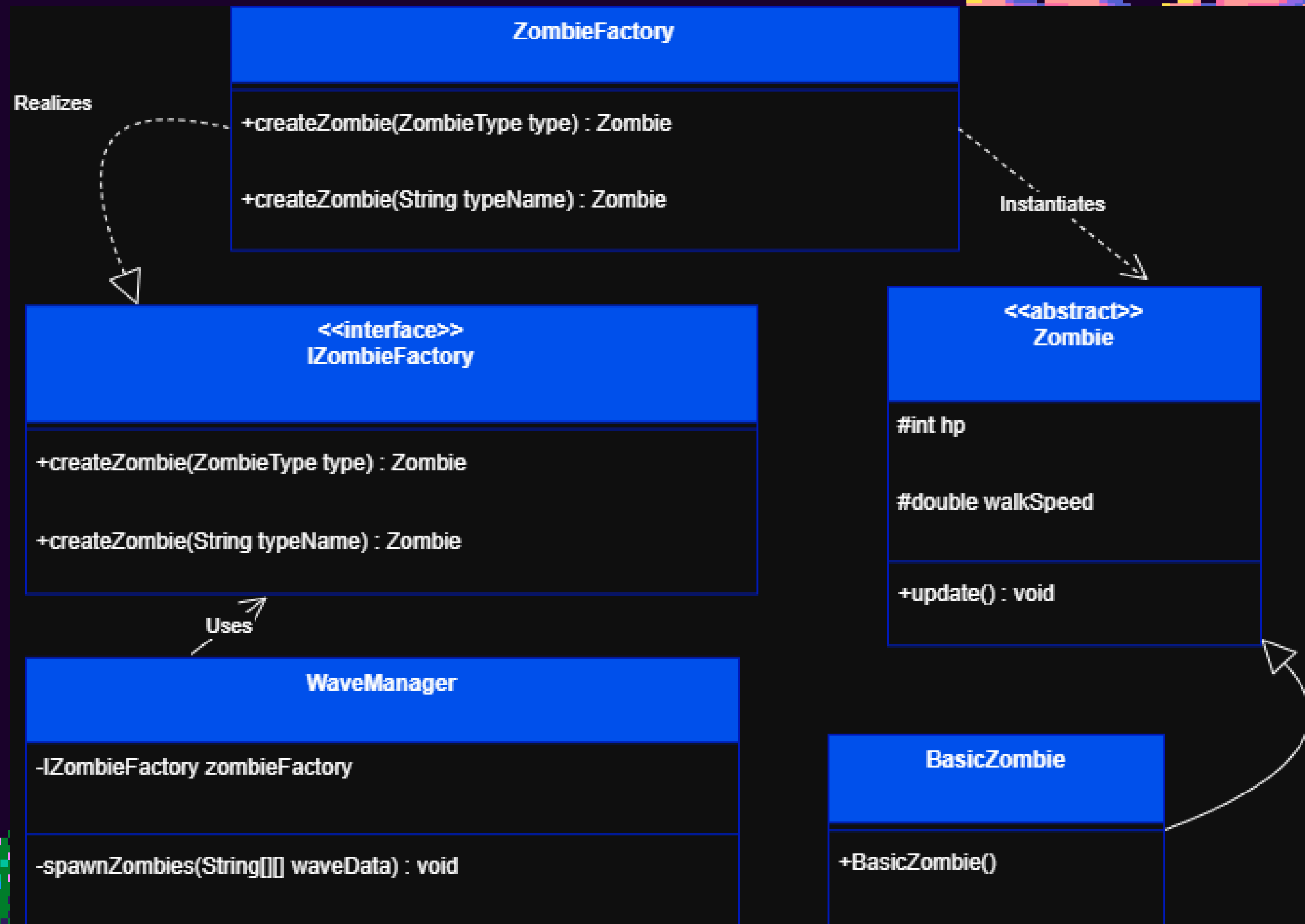


HOW IS THE
CODE
ORGANIZED?

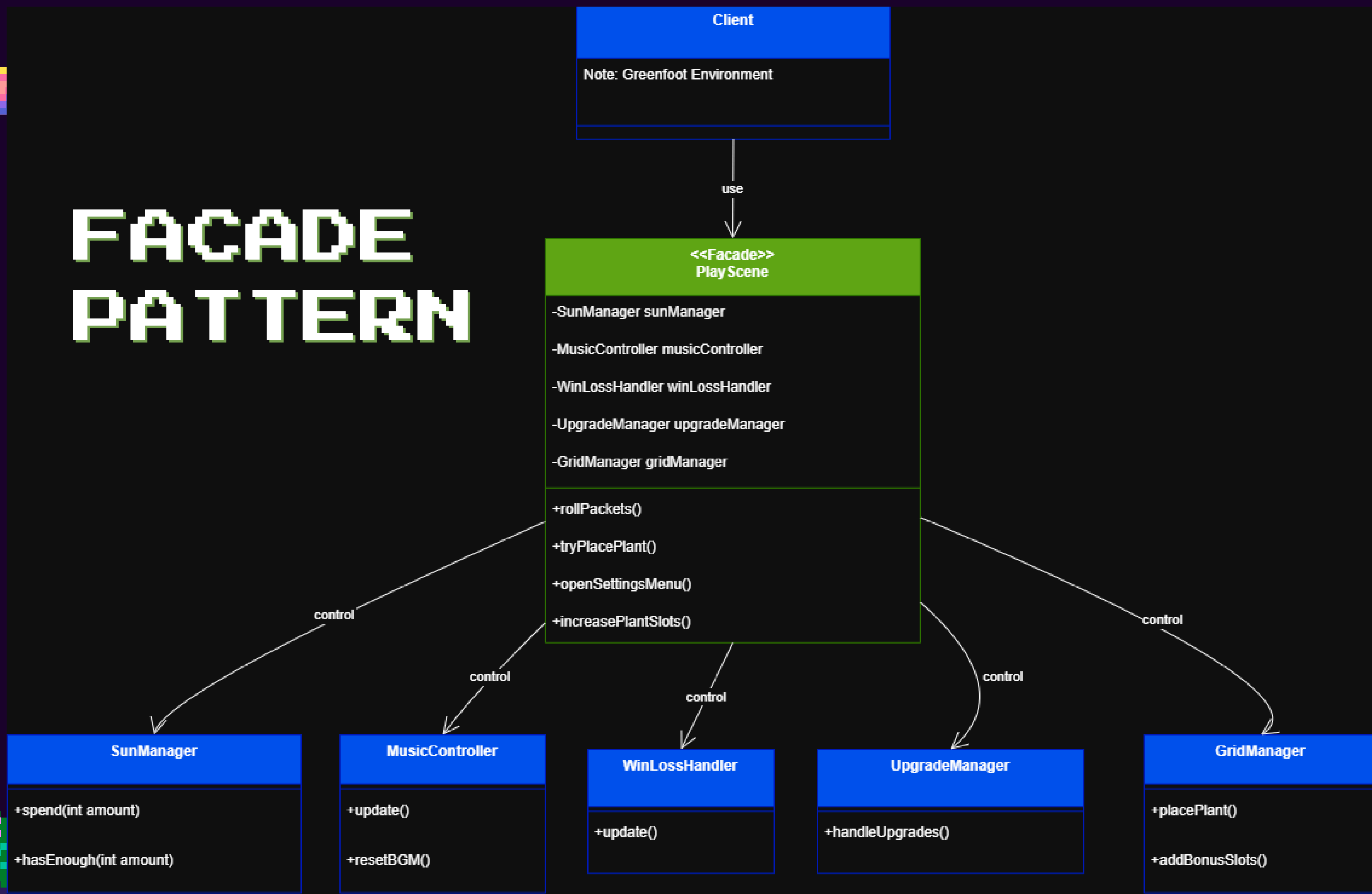
FACTORY METHOD



FACTORY METHOD



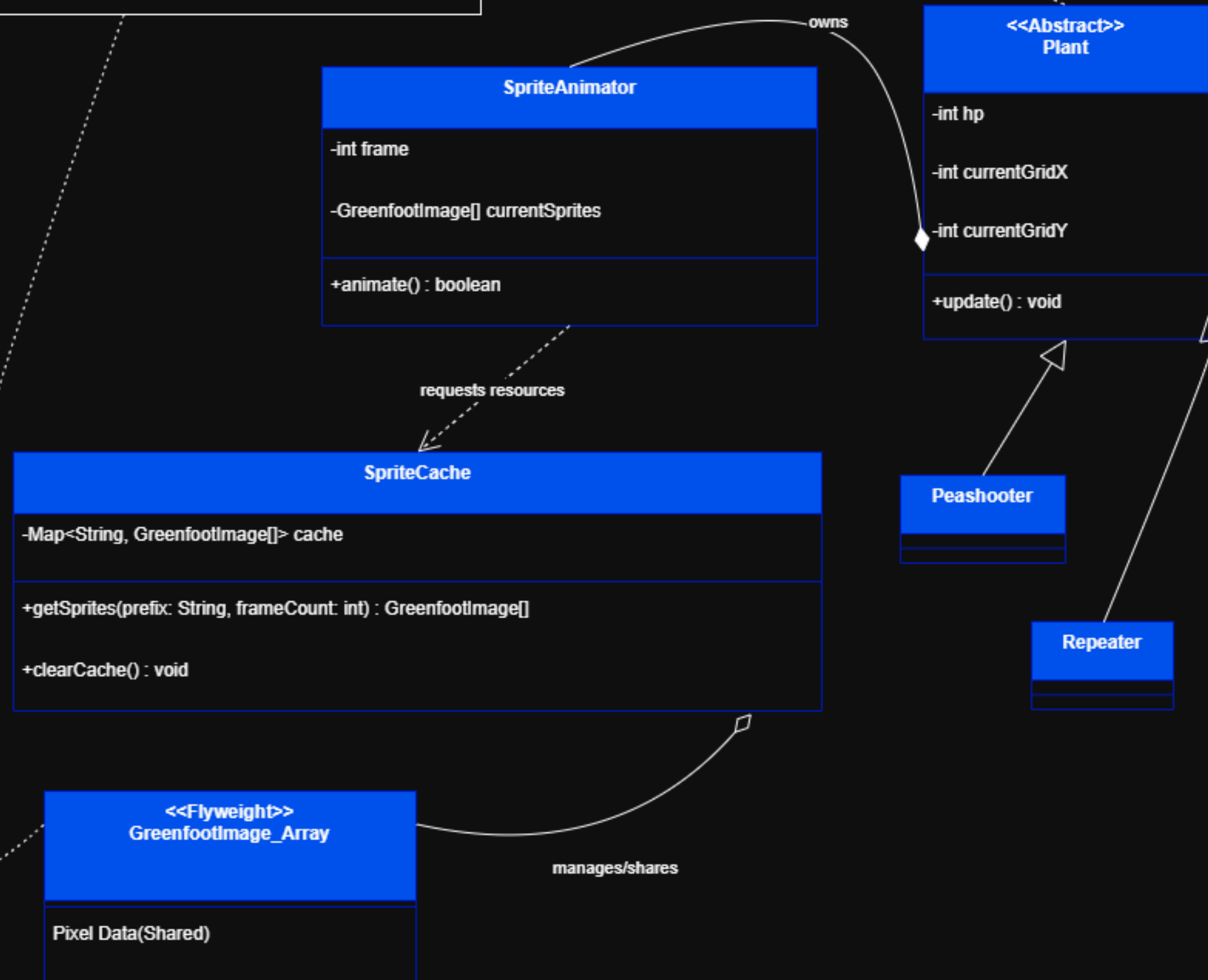
FACADE PATTERN



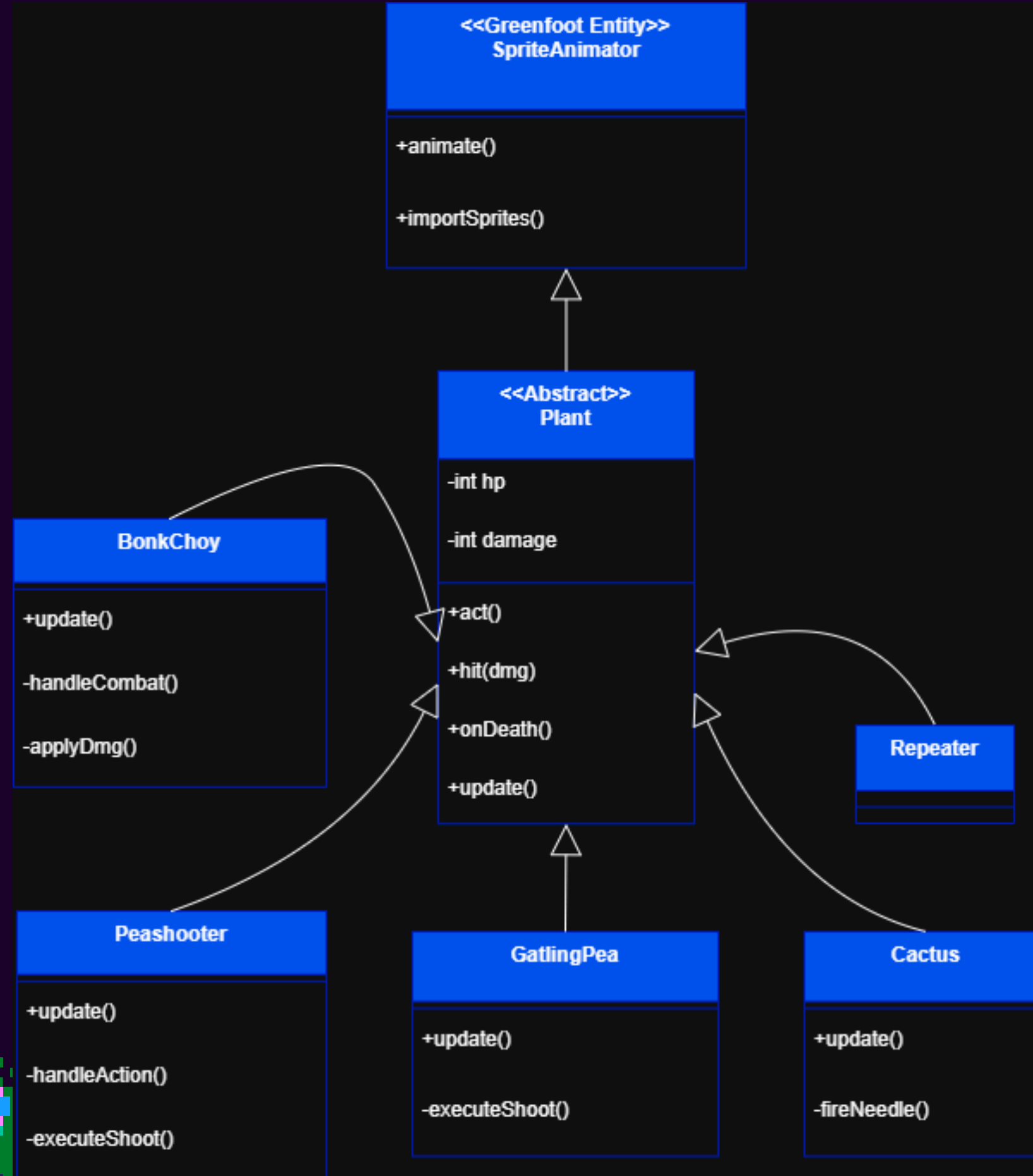
FLYWEIGHT PATTERN

Intrinsic State: Shared animation data that remains constant.

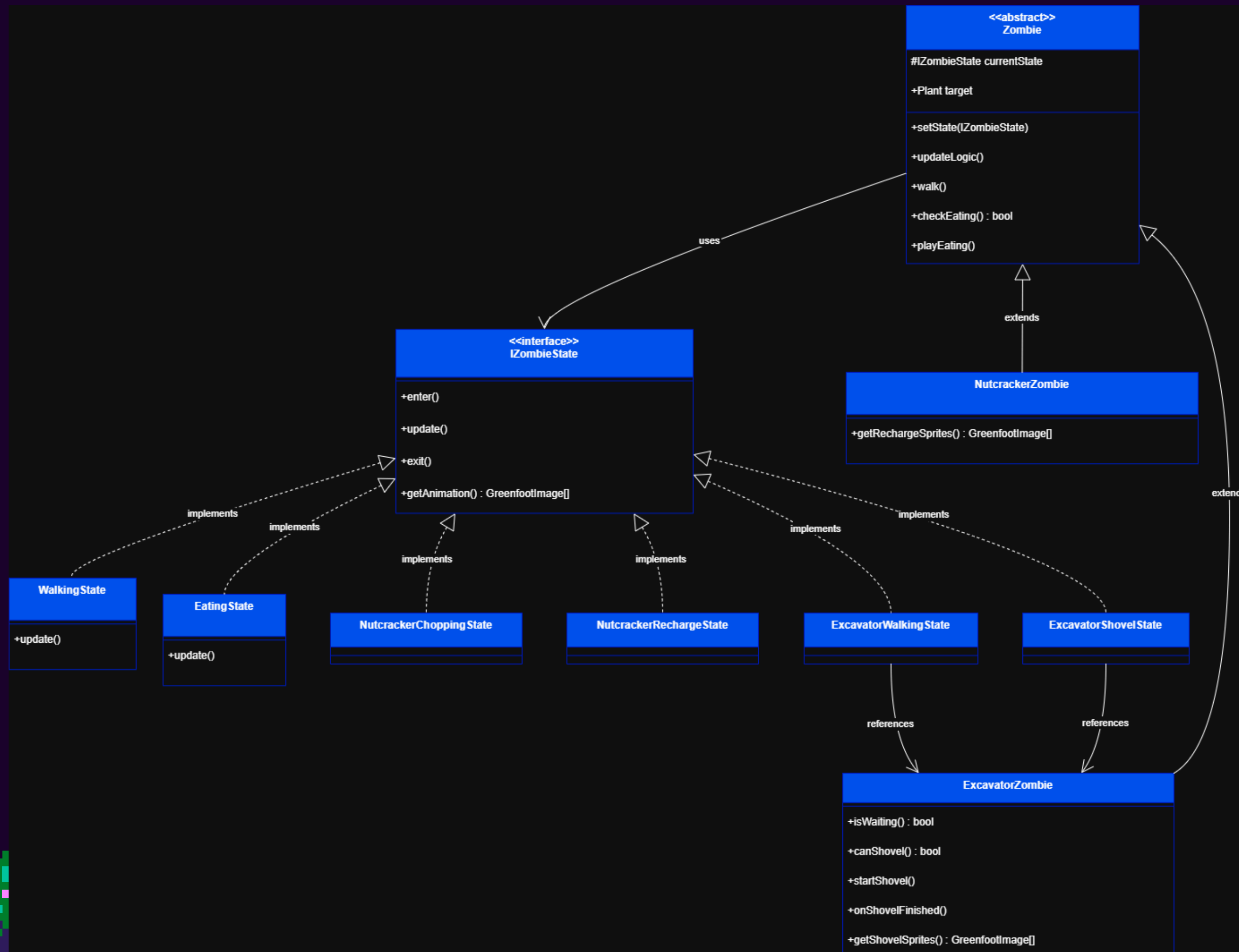
Extrinsic State: Individual stats like HP and grid position.



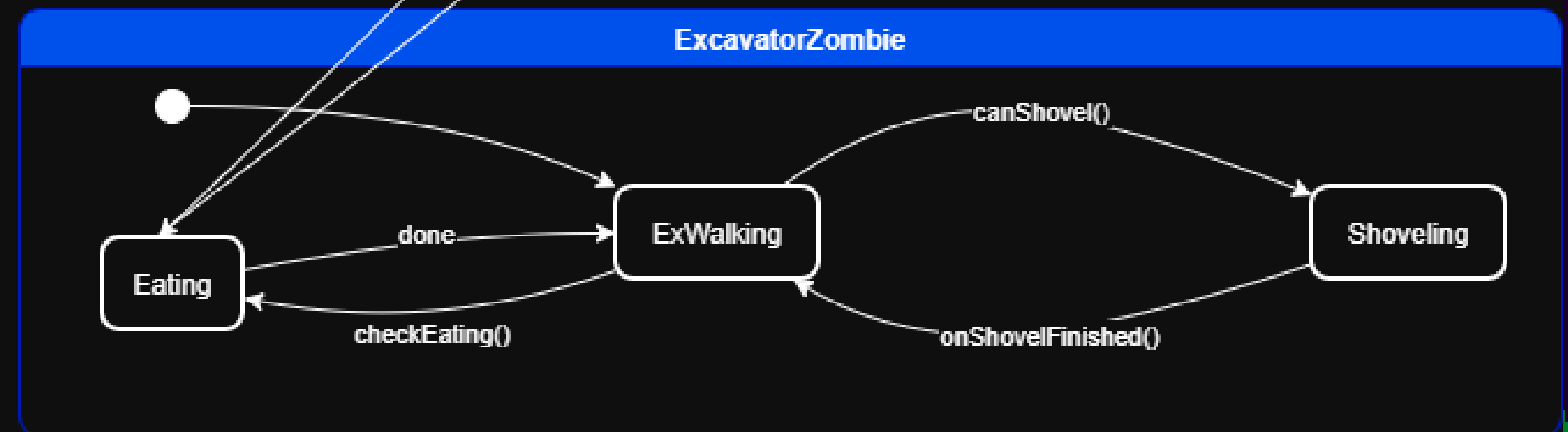
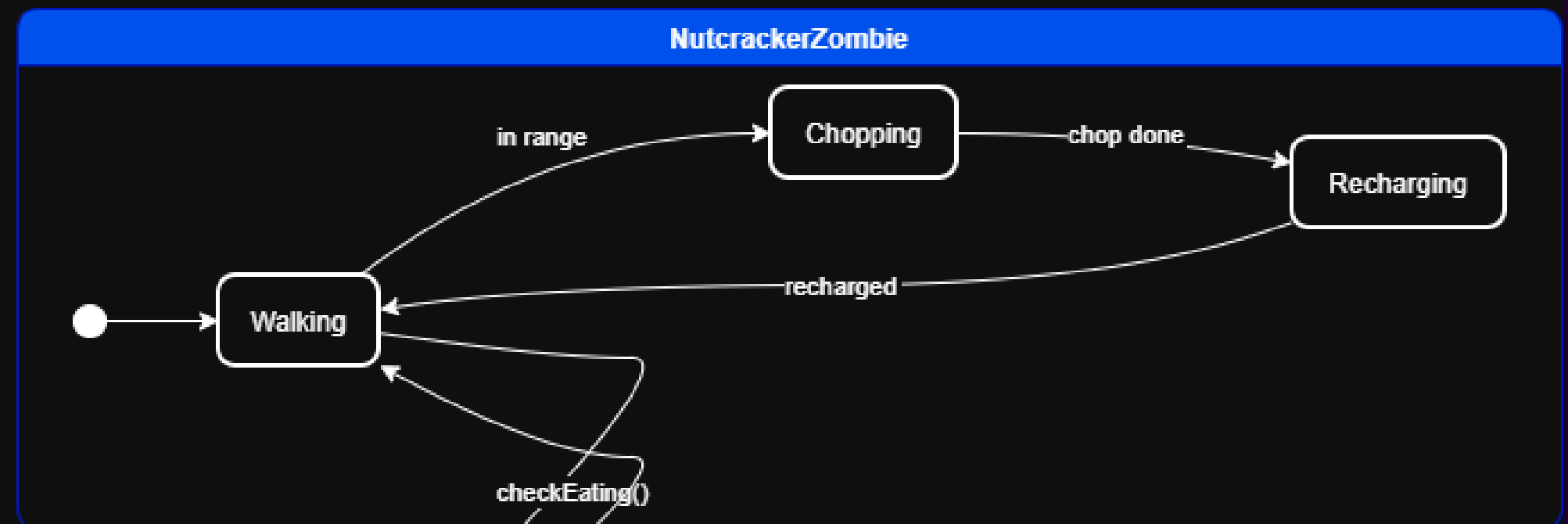
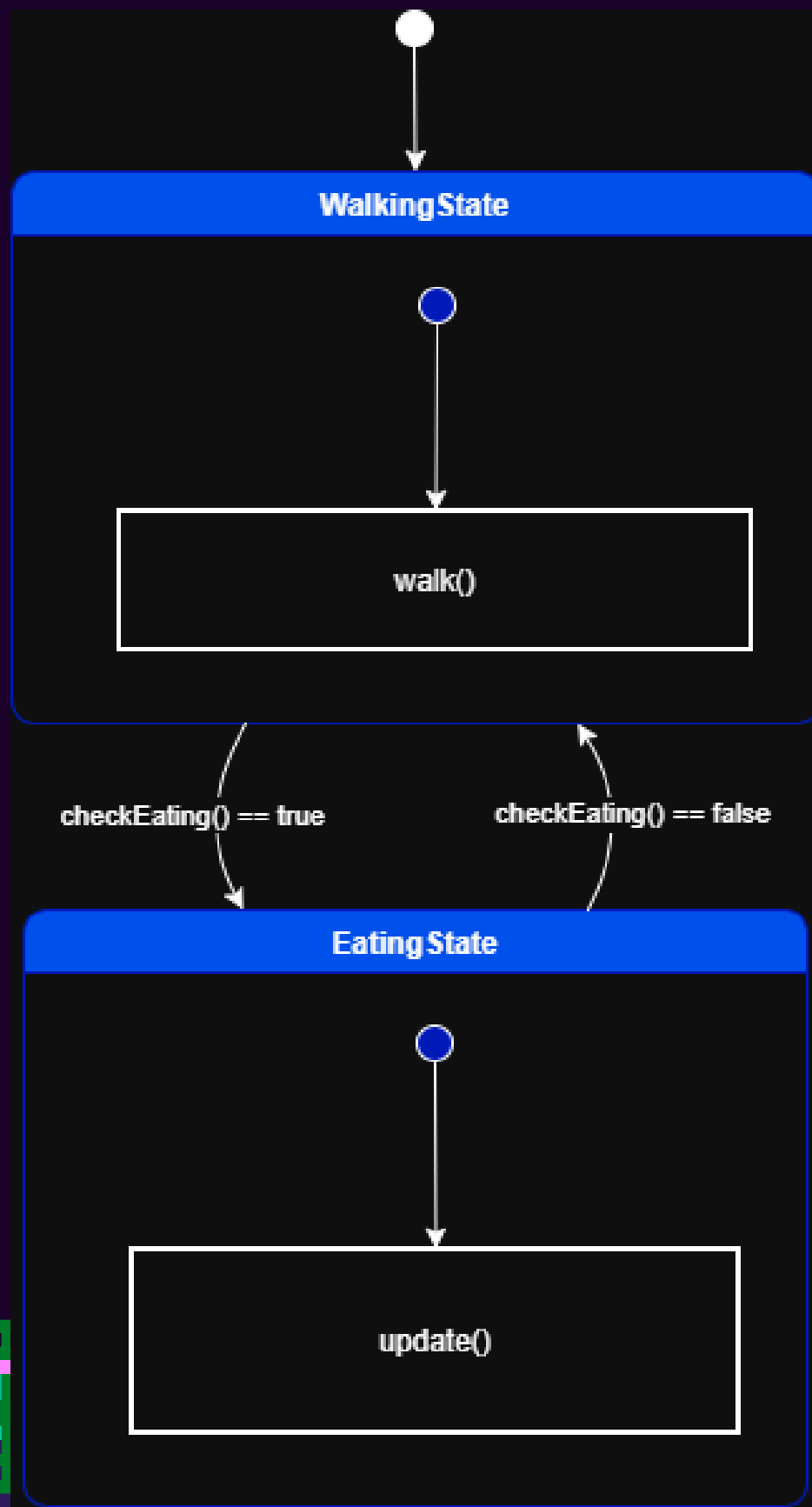
TEMPLATE METHOD



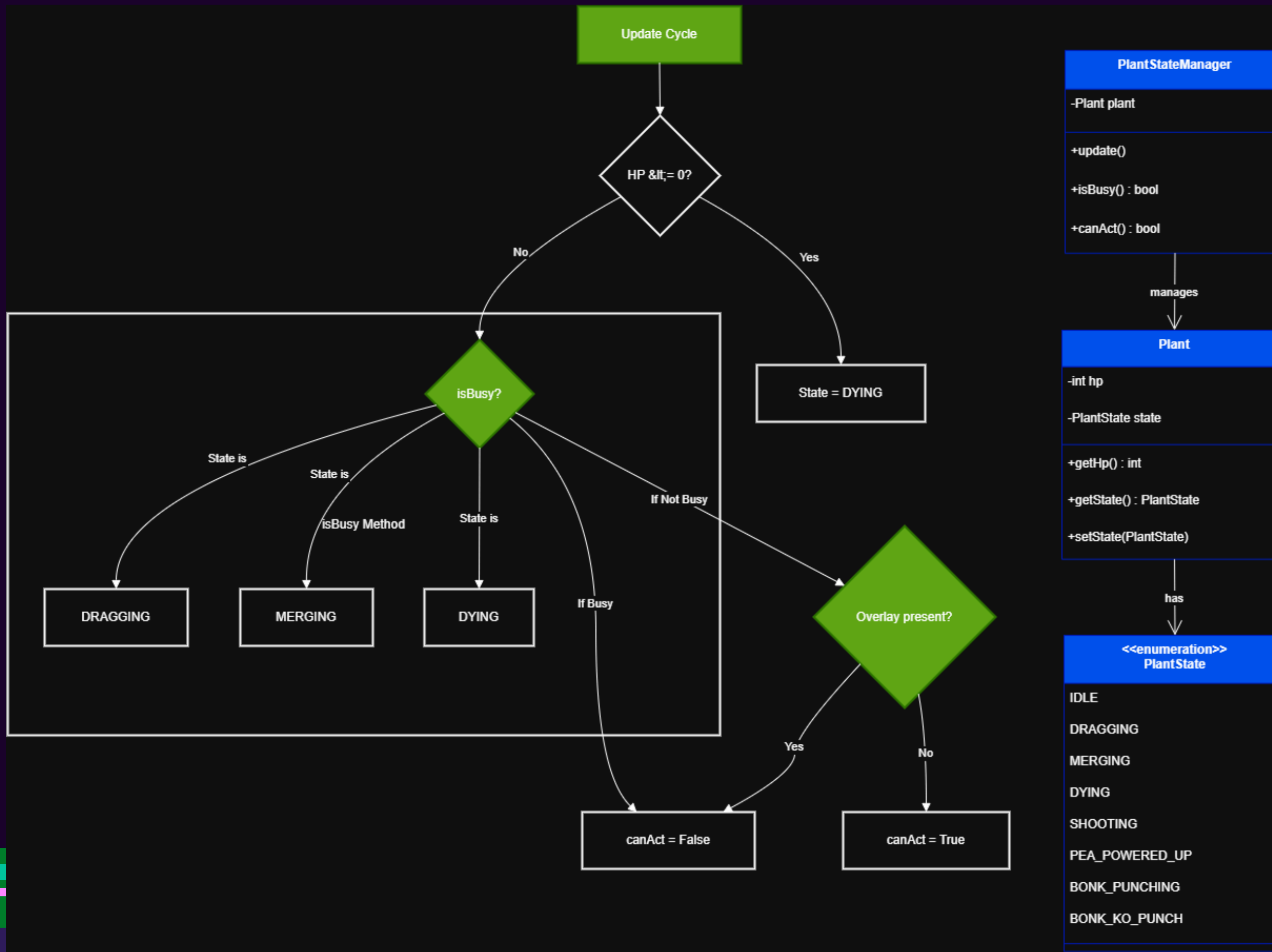
STATE PATTERN



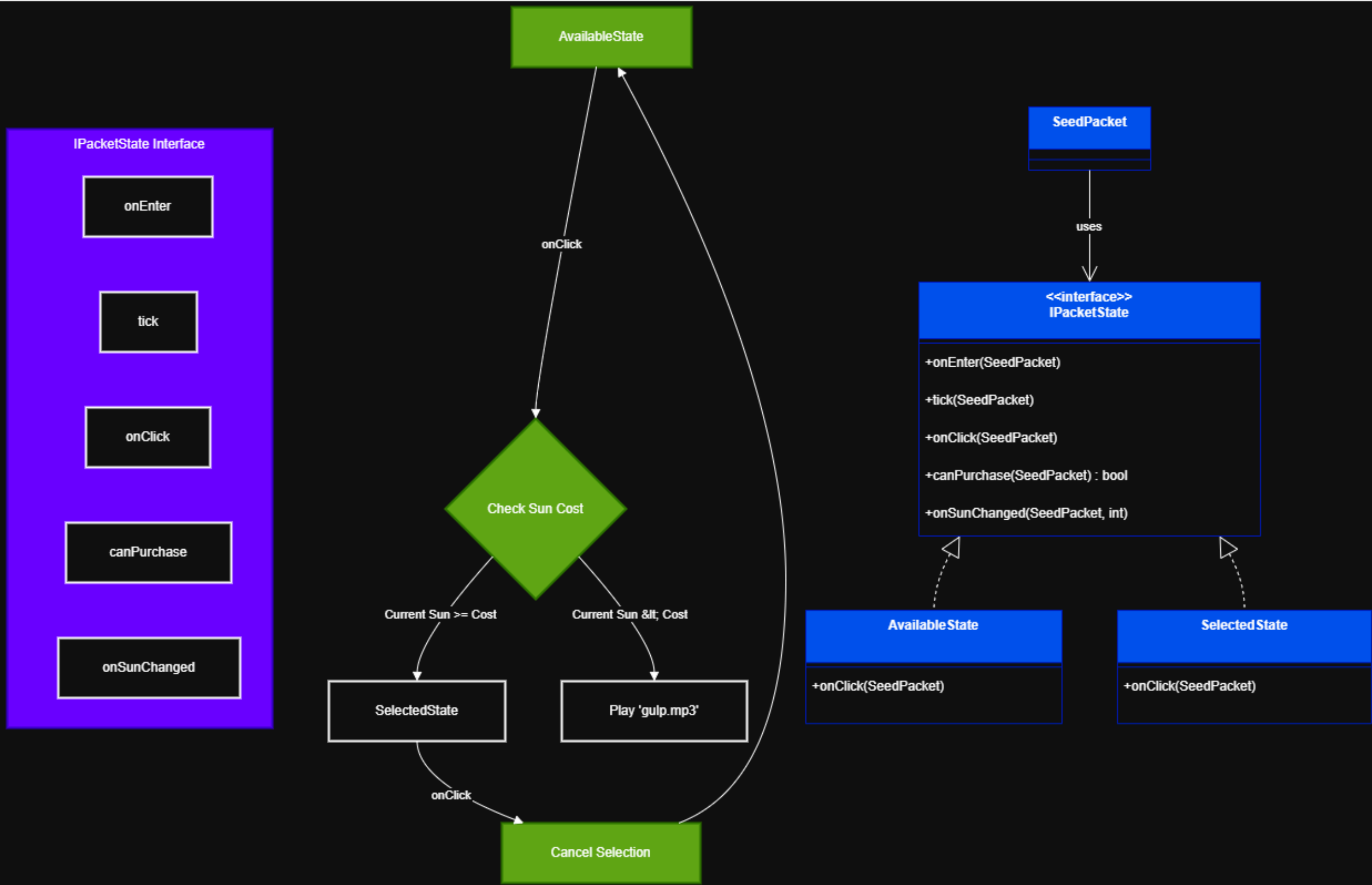
STATE PATTERN



STATE PATTERN

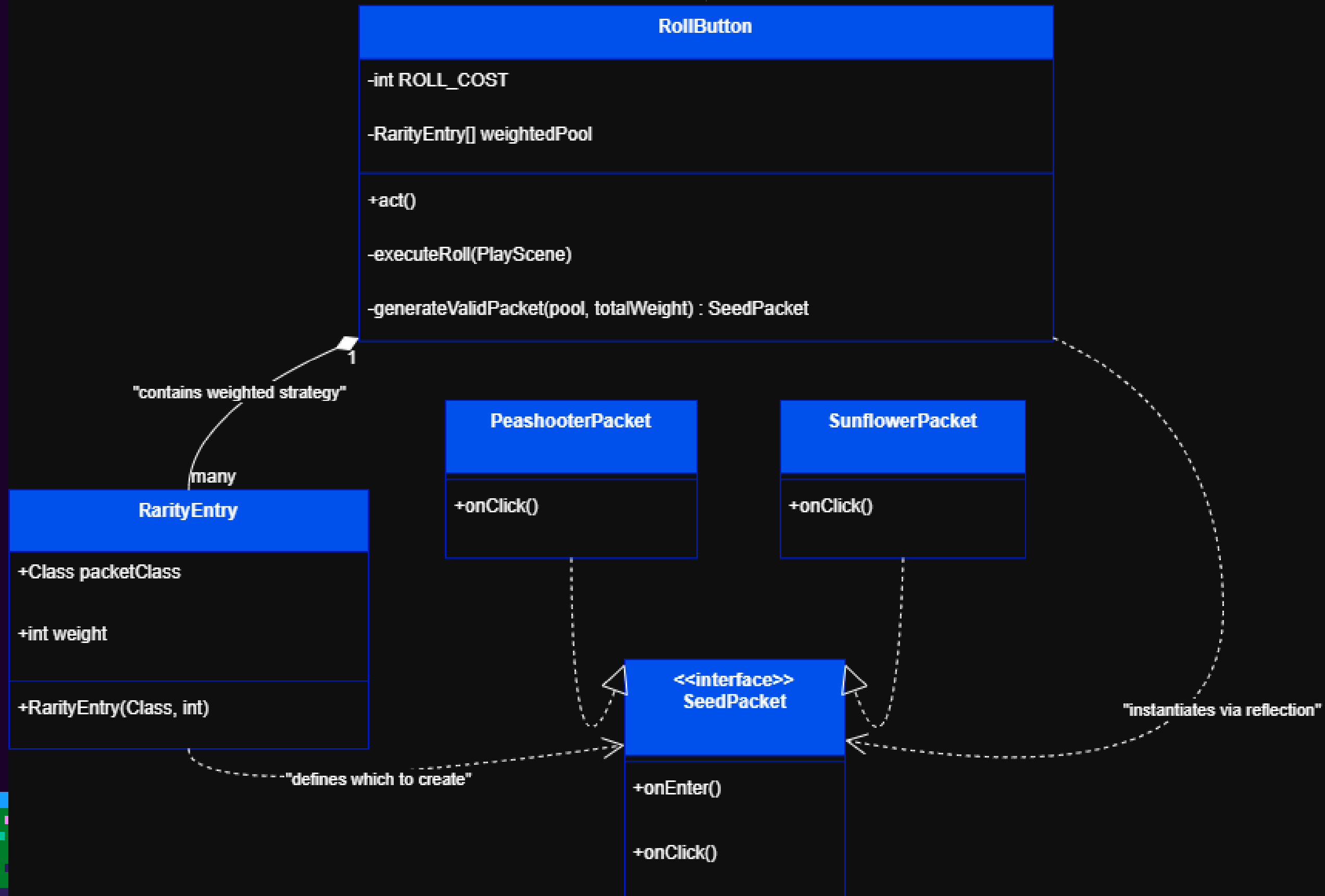


STATE PATTERN



STRATEGY PATTERN

generateValidPacket() uses the weightedPool strategy to pick a SeedPacket type



STRATEGY PATTERN

```
public RarityEntry[] weightedPool = {  
    new RarityEntry(BonkchoyPacket.class, 1),  
    new RarityEntry(PeashooterPacket.class, 3),  
    new RarityEntry(CactusPacket.class, 3),  
    new RarityEntry(RepeaterPacket.class, 0)  
};
```

STRATEGY PATTERN

```
if (isFree || sm.hasEnough(ROLL_COST)) {  
    RupButton.RarityEntry[] currentPool = world.rupbutton.getPoolForRoll();  
    int totalWeight = 0;  
    for (RupButton.RarityEntry entry : currentPool) {  
        if (entry != null) totalWeight += entry.weight;  
    }  
}
```

STRATEGY PATTERN

```
private SeedPacket generateValidPacket(RupButton.RarityEntry[] pool, int totalWeight) {  
    while (true) {  
        int randomNumber = Greenfoot.getRandomNumber(totalWeight);  
        int cursor = 0;  
        for (RupButton.RarityEntry entry : pool) {  
            if (entry == null) continue;  
            cursor += entry.weight;  
  
            if (randomNumber < cursor) {  
                try {  
                    return (SeedPacket) entry.packetClass.getDeclaredConstructor().newInstance();  
                } catch (Exception e) {  
                    e.printStackTrace();  
                }  
                break;  
            }  
        }  
    }  
}
```



Share...

- PlantState
- PlantStateManager
- PlantType
- Random
- SelectedState
- SpriteCache
- SunManager
- SunSpawner
- Timer1
- UpgradeManager
- WalkingState
- WaveState
- WinLossHandler
- ZombieAssets
- ZombieConfig
- ZombieDeathHandler
- ZombieEventBus
- ZombieFactory
- ZombieRegistry
- ZombieType

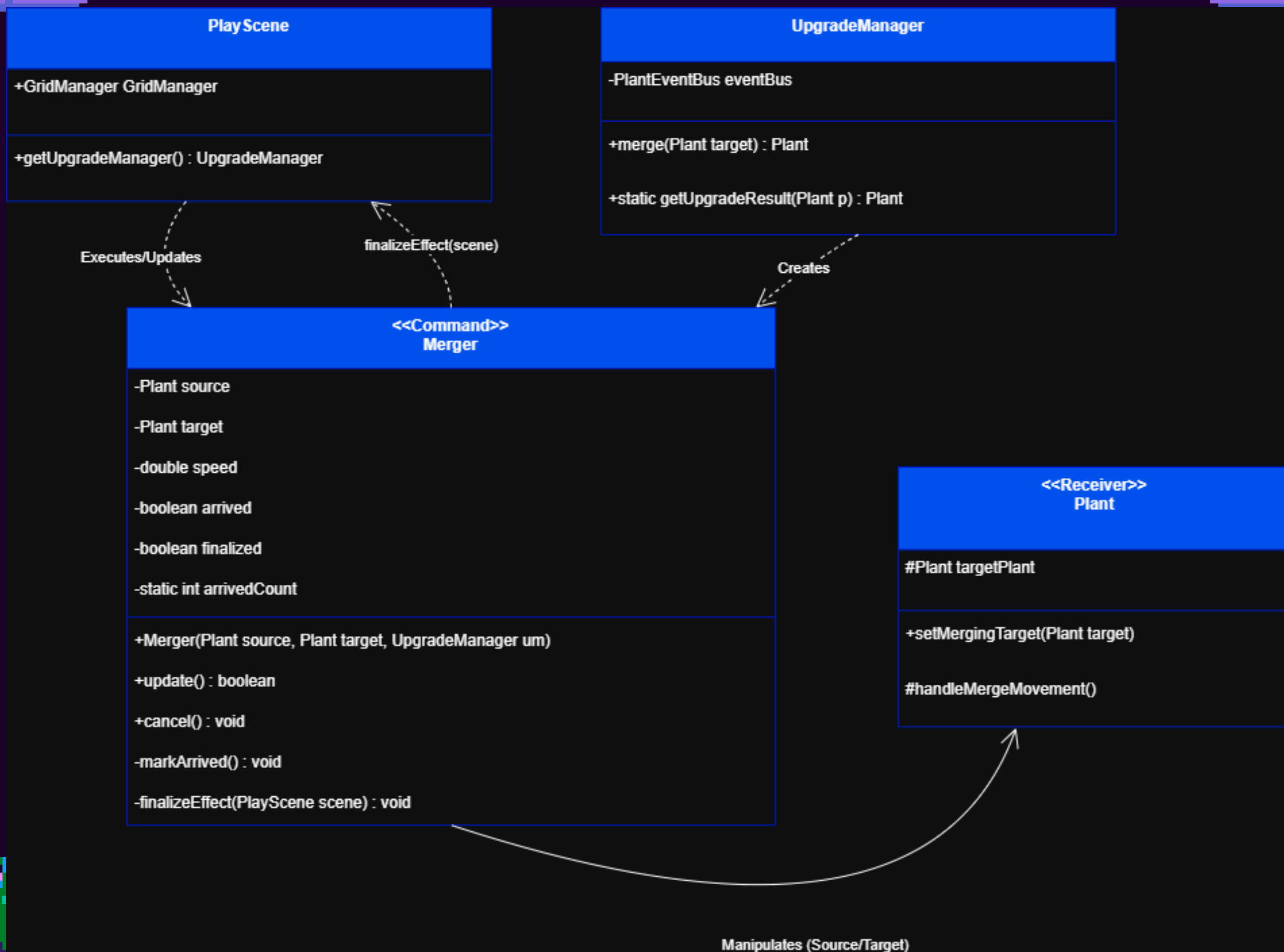
> Act

|| Pause

↺ Reset

Speed:

COMMAND PATTERN





OLL SEEDS
25



100475



Pause

Reset

Speed:



OBSERVER PATTERN



OBSERVER PATTERN

```
import greenfoot.*;  
import java.util.List;  
  
public abstract class Zombie extends SpriteAnimator implements IDamageable, IGridObject {  
    protected ZombieEventBus eventBus;
```

```
@Override  
public void hit(int dmg) {  
    if (!isAlive) return;  
    this.hp -= dmg;  
    eventBus.publishHit(this, dmg);  
    if (this.hp <= 0) {  
        this.hp = 0;  
        this.isAlive = false;  
    }  
}
```

```
import greenfoot.*;
```

```
public class HealthBar extends Actor implements IZombieEventListener, IPlantEventListener {
```

```
public HealthBar(Actor owner, int width) {
```

```
    this.owner = owner;
```

```
    this.width = width;
```

```
    // Subscribe vào EventBus ngay khi tạo
```

```
    if (owner instanceof Zombie) {
```

```
        ((Zombie) owner).eventBus.subscribe(this);
```

```
    } else if (owner instanceof Plant) {
```

```
        // subscribe sau khi addToWorld vì eventBus của Plant set sau
```

```
    }
```

```
}
```

```
@Override
```

```
public void onZombieHit(Zombie zombie, int damage) {
```

```
    cachedHp = zombie.getHp();
```

```
    cachedMaxHp = zombie.config.maxHp;
```

```
    lastDamageTime = System.currentTimeMillis();
```

```
    oldHp = cachedHp;
```

```
    redraw();
```

```
}
```





- 1.FACTORY METHOD
- 2.FACADE PATTERN
- 3.FLYWEIGHT PATTERN
- 4.TEMPLATE METHOD
- 5.STATE PATTERN
- 6.STRATEGY PATTERN
- 7.COMMAND PATTERN
- 8.OBSERVER PATTERN



THIS IS THE END.
THANKS!!!